Concept Bottleneck Models

Pang Wei Koh [*1] **Thao Nguyen** [*1 2] **Yew Siang Tang** [*1]
Stephen Mussmann [1] **Emma Pierson** [1] **Been Kim** [2] **Percy Liang** [1]

Abstract

We seek to learn models that we can interact with using high-level concepts: if the model did not think there was a bone spur in the x-ray, would it still predict severe arthritis? State-of-the-art models today do not typically support the manipulation of concepts like "the existence of bone spurs", as they are trained end-to-end to go directly from raw input (e.g., pixels) to output (e.g., arthritis severity). We revisit the classic idea of first predicting concepts that are provided at training time, and then using these concepts to predict the label. By construction, we can intervene on these *concept bottleneck models* by editing their predicted concept values and propagating these changes to the final prediction. On x-ray grading and bird identification, concept bottleneck models achieve competitive accuracy with standard end-to-end models, while enabling interpretation in terms of high-level clinical concepts ("bone spurs") or bird attributes ("wing color"). These models also allow for richer human-model interaction: accuracy improves significantly if we can correct model mistakes on concepts at test time.

1. Introduction

Suppose that a radiologist is collaborating with a machine learning model to grade the severity of knee osteoarthritis. She might ask why the model made its prediction—did it deem the space between the knee joints too narrow? Or she might seek to intervene on the model—if she told it that the x-ray showed a bone spur, would its prediction change?

State-of-the-art models today do not typically support such queries: they are end-to-end models that go directly from raw input x (e.g., pixels) to target y (e.g., arthritis severity), and we cannot easily interact with them using the same high-level concepts that practitioners reason with, like "joint space narrowing" or "bone spurs".

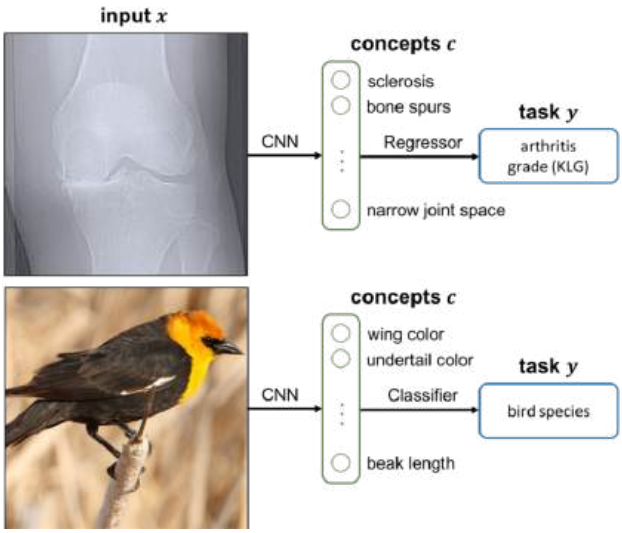


Figure 1. We study concept bottleneck models that first predict an intermediate set of human-specified concepts c, then use c to predict the final output y. We illustrate the two applications we consider: knee x-ray grading and bird identification.

We approach this problem by revisiting the simple, classic idea of first predicting an intermediate set of human-specified concepts c like "joint space narrowing" and "bone spurs", then using c to predict the target y. In this paper, we refer to such models as *concept bottleneck models*. These models are trained on data points (x, c, y), where the input x is annotated with both concepts c and target y. At test time, they take in an input x, predict concepts $\hat{c}$, and then use those concepts to predict the target $\hat{y}$ (Figure 1).

Earlier versions of concept bottleneck models were overtaken in predictive accuracy by end-to-end neural networks (e.g., Kumar et al. (2009) for face recognition and Lampert et al. (2009) for animal identification), leading to a perceived tradeoff between accuracy and interpretability in terms of concepts. In this paper, we propose a straightforward method for turning any end-to-end neural network into a concept bottleneck model, given concept annotations at training time: we simply resize one of the layers

[*]Equal contribution [1]Stanford University [2]Google Research. Correspondence to: Pang Wei Koh <pangwei@cs.stanford.edu>, Been Kim <beenkim@google.com>, Percy Liang <pliang@cs.stanford.edu>.

Proceedings of the 37th International Conference on Machine Learning, Online, PMLR 119, 2020. Copyright 2020 by the author(s).

to match the number of concepts provided, and add an intermediate loss that encourages the neurons in that layer to align component-wise to the provided concepts. We show that concept bottleneck models trained in this manner can achieve task accuracies competitive with or even higher than standard models. We emphasize that concept annotations are not needed at test time; the model predicts the concepts, then uses the predicted concepts to make a final prediction.

Importantly—and unlike standard end-to-end models—these bottleneck models support interventions on concepts: we can edit the concept predictions $\hat{c}$ and propagate those changes to the target prediction $\hat{y}$. Interventions enable richer human-model interaction: e.g., if the radiologist realizes that what the model thinks is a bone spur is actually an artifact, she can update the model's prediction by directly changing the corresponding value of $\hat{c}$. When we simulate this injection of human knowledge by partially correcting concept mistakes that the model makes at test time, we find that accuracy improves substantially beyond that of a standard model.

Interventions also make concept bottleneck models interpretable in terms of high-level concepts: by manipulating concepts $\hat{c}$ and observing the model's response, we can obtain counterfactual explanations like "if the model did not think the joint space was too narrow for this patient, then it would not have predicted severe arthritis". In contrast, prior work on explaining end-to-end models in terms of high-level concepts has been restricted to post-hoc interpretation of already-trained end-to-end models: for example, predicting concepts from hidden layers (Kim et al., 2018) or measuring the correlation of individual neurons with concepts (Bau et al., 2017).

The validity of interventions on a model depends on the alignment between its predicted concepts $\hat{c}$ and the true concepts c. We can estimate this alignment by measuring the model's concept accuracy on a held-out validation set (Fong & Vedaldi, 2017).[1] A model with perfect concept accuracy across all possible inputs makes predictions $\hat{c}$ that align with the true concepts c. Conversely, if a model has low concept accuracy, then the model's predictions $\hat{c}$ need not match with the true concepts, and we would not expect interventions to lead to meaningful results.

Contributions. We systematically study variants of concept bottleneck models and contrast them with standard end-to-end models in different settings, with a focus on the previously-unexplored ability of concept bottleneck models to support concept interventions. Our goal is to characterize concept bottleneck models more fully: Is there a tradeoff between task accuracy and concept interpretability? Do interventions at test time help model accuracy, and is concept accuracy a good indicator of the ability to effectively intervene? Do different ways of training bottleneck models lead to significantly different outcomes in intervention?

We evaluate concept bottleneck models on the two applications in Figure 1: the osteoarthritis grading task (Nevitt et al., 2006) and a fine-grained bird species identification task (Wah et al., 2011). On these tasks, we show that bottleneck models are competitive with standard end-to-end models while also attaining high concept accuracies. In contrast, the concepts cannot be predicted with high accuracy from linear combinations of neurons in a standard black-box model, making it difficult to do post-hoc interpretation in terms of concepts like in Kim et al. (2018). We demonstrate that we can substantially improve model accuracy by intervening on these bottleneck models at test time to correct model mistakes on concepts. Finally, we show that bottleneck models guided to learn the right concepts can also be more robust to covariate shifts.

2. Related work

Concept bottleneck models. Models that bottleneck on human-specified concepts—where the model first predicts the concepts, then uses only those predicted concepts to make a final prediction—have been previously used for specific applications (Kumar et al., 2009; Lampert et al., 2009). Early versions did not use end-to-end neural networks, which soon overtook them in predictive accuracy. Consequently, bottleneck models have historically been more popular for few-shot learning settings, where shared concepts might allow generalization to unseen contexts, rather than the standard supervised setting we consider here.

More recently, deep neural networks with concept bottlenecks have re-emerged as targeted tools for solving particular tasks, e.g., Fauw et al. (2018) for retinal disease diagnosis, Yi et al. (2018) for visual question-answering, and Bucher et al. (2018) for content-based image retrieval. Losch et al. (2019) and Chen et al. (2020) also explore learning concept-based models via auxiliary datasets.

Concept bottlenecks differ from traditional feature engineering: we learn mappings from raw input to high-level concepts, whereas feature engineering constructs low-level features that can be computed by handwritten functions.

Concepts as auxiliary losses or features. Non-bottleneck models that use human-specified concepts commonly use them in auxiliary objectives in a multi-task setup, or as auxiliary features; examples include using object parts (Huang et al., 2016; Zhou et al., 2018), parse trees (Zelenko et al., 2003; Bunescu & Mooney, 2005), or natural language explanations (Murty et al., 2020). However, these models do not support intervention on concepts. For instance, consider

[1] With the usual caveats of measuring accuracy: in practice, the validation set might be skewed such that models that learn spurious correlations can still achieve high concept accuracy.

a multi-task model $c \leftarrow x \rightarrow y$, with the concepts c used in an auxiliary loss; simply intervening on $\hat{c}$ at test time will not affect the model's prediction of y. Interventions do affect models that use c as auxiliary features by first predicting $x \rightarrow c$ and then predicting $(x, c) \rightarrow y$ (e.g., Sutton & McCallum (2005)), but we cannot intervene in isolation on a single concept because of the side channel from $x \rightarrow y$.

Causal models. There has been extensive work on learning models of causal relationships in the world (Pearl, 2000). While concept bottleneck models can represent causal relationships between $x \rightarrow c \rightarrow y$ if the set of concepts c is chosen appropriately, they are flexible and do not require c to cause y. This is an advantage in settings where it is difficult to fully specify the causal $c \rightarrow y$ graph. For example, imagine that arthritis grade (y) is highly correlated with swelling (c). In this case, c does not cause y (hypothetically, if one could directly induce swelling in the patient, it would not affect whether they had osteoarthritis). However, concept bottleneck models can still exploit the fact that c is highly predictive for y, and intervening on the model by replacing the predicted concept value $\hat{c}$ with the true value c can still improve accuracy, even if c does not cause y.

A central claim of this paper is that we can intervene on concept bottleneck models. Intervention is a standard notion in causal inference, and we emphasize that we intervene on the value of a predicted concept within the model, not on that concept in reality. In other words, we are interested in how changing the model's predicted concept values $\hat{c}$ would affect its final prediction $\hat{y}$, and not whether intervening on the true concept value c in reality (e.g., by inducing swelling) would affect the true label y. See, e.g., Goyal et al. (2019); O'Shaughnessy et al. (2020) for discussions on causality in the context of interpreting models with concepts.

Post-hoc concept analysis. Many methods have been developed to interpret models post-hoc, including recent work on using human-specified concepts to generate explanations (Bau et al., 2017; Kim et al., 2018; Zhou et al., 2018; Ghorbani et al., 2019). These techniques rely on models automatically learning those concepts despite not having explicit knowledge of them, and can be particularly useful when paired with models that attempt to learn more interpretable representations (Bengio et al., 2013; Chen et al., 2016; Higgins et al., 2017; Melis & Jaakkola, 2018). However, post-hoc methods can fail when the models do not learn these concepts, and also do not admit straightforward interventions on concepts. In this work, we instead directly guide models to learn these concepts at training time.

3. Setup

Consider predicting a target $y \in \mathbb{R}$ from input $x \in \mathbb{R}^d$; for simplicity, we present regression first and discuss classification later. We observe training points $\{(x^{(i)}, y^{(i)}, c^{(i)})\}_{i=1}^n$, where $c \in \mathbb{R}^k$ is a vector of k concepts. We consider bottleneck models of the form $f(g(x))$, where $g : \mathbb{R}^d \rightarrow \mathbb{R}^k$ maps an input x into the concept space ("bone spurs", etc.), and $f : \mathbb{R}^k \rightarrow \mathbb{R}$ maps concepts into a final prediction ("arthritis severity"). We call these *concept bottleneck models* because their prediction $\hat{y} = f(g(x))$ relies on the input x entirely through the bottleneck $\hat{c} = g(x)$, which we train to align component-wise to the concepts c. We define *task accuracy* as how accurately $f(g(x))$ predicts y, and *concept accuracy* as how accurately $g(x)$ predicts c (averaged over each concept). We will refer to $g(\cdot)$ as predicting $x \rightarrow c$, and to $f(\cdot)$ as predicting $c \rightarrow y$.

In our work, we systematically study different ways of learning concept bottleneck models. Let $L_{C_j} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}_+$ be a loss function that measures the discrepancy between the predicted and true j-th concept, and let $L_Y : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}_+$ measure the discrepancy between predicted and true targets. We consider the following ways to learn a model $(\hat{f}, \hat{g})$:

1. The *independent bottleneck* learns $\hat{f}$ and $\hat{g}$ independently: $\hat{f} = \arg\min_f \sum_i L_Y(f(c^{(i)}); y^{(i)})$, and $\hat{g} = \arg\min_g \sum_{i,j} L_{C_j}(g_j(x^{(i)}); c_j^{(i)})$. While $\hat{f}$ is trained using the true c, at test time it still takes $\hat{g}(x)$ as input.

2. The *sequential bottleneck* first learns $\hat{g}$ in the same way as above. It then uses the concept predictions $\hat{g}(x)$ to learn $\hat{f} = \arg\min_f \sum_i L_Y(f(\hat{g}(x^{(i)})); y^{(i)})$.

3. The *joint bottleneck* minimizes the weighted sum $\hat{f}, \hat{g} = \arg\min_{f,g} \sum_i \big[L_Y(f(g(x^{(i)})); y^{(i)}) + \sum_j \lambda L_{C_j}(g(x^{(i)}); c^{(i)})\big]$ for some $\lambda > 0$.

4. The *standard model* ignores concepts and directly minimizes $\hat{f}, \hat{g} = \arg\min_{f,g} \sum_i L_Y(f(g(x^{(i)})); y^{(i)})$.

The hyperparameter λ in the joint bottleneck controls the tradeoff between concept vs. task loss. The standard model is equivalent to taking $\lambda \rightarrow 0$, while the sequential bottleneck can be viewed as taking $\lambda \rightarrow \infty$.

We propose a simple scheme to turn an end-to-end neural network into a concept bottleneck model: simply resize one of its layers to have k neurons to match the number of concepts k, then choose one of the training schemes above.

Classification. In classification, f and g compute real-valued scores (e.g., concept logits $\hat{\ell} = \hat{g}(x) \in \mathbb{R}^k$) that we then turn into a probabilistic prediction (e.g., $P(\hat{c}_j = 1) = \sigma(\hat{\ell}_j)$ for logistic regression). This does not change the independent bottleneck, since f (the $c \rightarrow y$ model) is directly trained on the binary-valued c. For the sequential and joint bottlenecks, we connect $c \rightarrow y$ to the logits $\hat{\ell}$, i.e., we compute $P(\hat{c}_j = 1) = \sigma(\hat{g}_j(x))$ and $P(\hat{y} = 1) = \sigma(\hat{f}(\hat{g}(x)))$.

Table 1. Task errors with ±2SD over random seeds. Overall, concept bottleneck models are competitive with standard models.

MODEL	y RMSE (OAI)	y ERROR (CUB)
INDEPENDENT	0.435± 0.024	0.240±0.012
SEQUENTIAL	0.418± 0.004	0.243±0.006
JOINT	0.418± 0.004	0.199±0.006
STANDARD	0.441± 0.006	0.175±0.008
NO BOTTLENECK	0.443 ± 0.008	0.173±0.003
MULTITASK	0.425± 0.010	0.162±0.002

Table 2. Average concept errors. Bottleneck models have lower error than linear probes on standard and SENN models.

MODEL	c RMSE (OAI)	c ERROR (CUB)
INDEPENDENT	0.529±0.004	0.034±0.002
SEQUENTIAL	0.527±0.004	0.034±0.002
JOINT	0.543±0.014	0.031±0.000
STANDARD [PROBE]	0.680±0.038	0.093±0.004
SENN [PROBE]	0.676±0.026	-

4. Benchmarking bottleneck model accuracy

We start by showing that concept bottleneck models achieve both competitive task accuracy and high concept accuracy. While this is necessary for bottleneck models to be viable in practice, their strength is that we can interpret and intervene on them; we explore those aspects in Sections 5 and 6.

4.1. Applications

We consider an x-ray grading and a bird identification task. Their corresponding datasets are annotated with high-level concepts that practitioners (radiologists/birders) use to reason about their decisions. (Dataset details in Appendix A.)

X-ray grading (OAI). We use knee x-rays from the Osteoarthritis Initiative (OAI) (Nevitt et al., 2006), which compiles radiological and clinical data on patients at risk of knee osteoarthritis (Figure 1-Top; $n = 36,369$ data points). Given an x-ray, the task is to predict the Kellgren-Lawrence grade (KLG), a 4-level ordinal variable assessed by radiologists that measures the severity of osteoarthritis, with higher scores denoting more severe disease.[2] As concepts, we use $k = 10$ ordinal variables describing joint space narrowing, bone spurs, calcification, etc.; these clinical concepts are also assessed by radiologists and used directly in the assessment of KLG (Kellgren & Lawrence, 1957).

Bird identification (CUB). We use the Caltech-UCSD Birds-200-2011 (CUB) dataset (Wah et al., 2011), which comprises $n = 11,788$ bird photographs (Figure 1-Bot). The task is to classify the correct bird species out of 200 possible options. As concepts, we use $k = 112$ binary bird attributes representing wing color, beak shape, etc. Because the provided concepts are noisy (see Appendix A), we denoise them by majority voting, e.g., if more than 50% of crows have black wings in the data, then we set all crows to have black wings. In other words, we use class-level concepts and assume that all birds of the same species in the training data share the same concept annotations. In contrast, the OAI dataset uses instance-level concepts: examples with the same y can have different concept annotations c.

[2]Due to technicalities in the data collection protocol, we use a modified version of KLG where the first two grades are combined.

Models. For each task, we construct concept bottleneck models by adopting model architectures and hyperparameters from previous high-performing approaches; see Appendix B for experimental details. For the joint bottleneck model, we search over the task-concept tradeoff hyperparameter λ and report results for the model that has the highest task accuracy while maintaining high concept accuracy on the validation set ($\lambda = 1$ for OAI and $\lambda = 0.01$ for CUB). We model x-ray grading as a regression problem (minimizing mean squared error) on both the KLG target y and concepts c, following Pierson et al. (2019); we fine-tune a pretrained ResNet-18 model to predict $x \to c$ (He et al., 2016), and use a small 3-layer MLP for $c \to y$. We model bird identification as multi-class classification for the species y and binary classification for the concepts c. Following Cui et al. (2018), we fine-tune an Inception-v3 network (Szegedy et al., 2016) to predict $x \to c$, and use a single linear layer (logistic regression) to predict $c \to y$.

4.2. Task and concept accuracies

Table 1 shows that concept bottleneck models achieve competitive task accuracy with standard black-box models on both tasks, despite the bottleneck constraint (all numbers reported are on a held-out test set). On OAI, joint and sequential bottlenecks are actually slightly better in root mean square error (RMSE) than the standard model,[3] and on CUB, sequential and independent bottlenecks are slightly worse in 0-1 error; all other models perform similarly. From Table 1, joint bottlenecks can do slightly better than sequential bottlenecks, which in turn can do better than independent bottlenecks, though this difference is not consistent. Compared to independent bottlenecks, sequential bottlenecks allow the $c \to y$ part of the model to adapt to how well it can predict $x \to c$; and joint bottlenecks further allow the model's version of the concepts to be refined to improve predictive performance.

At the same time, the bottleneck models are able to accurately predict each concept well (Figure 2), and they achieve low average error across all concepts (Table 2). As discussed

[3]To contextualize RMSE, our modified KLG ranges from 0-3, and average Pearson correlations between each predicted and true concept are ≥ 0.87 for all bottleneck models.

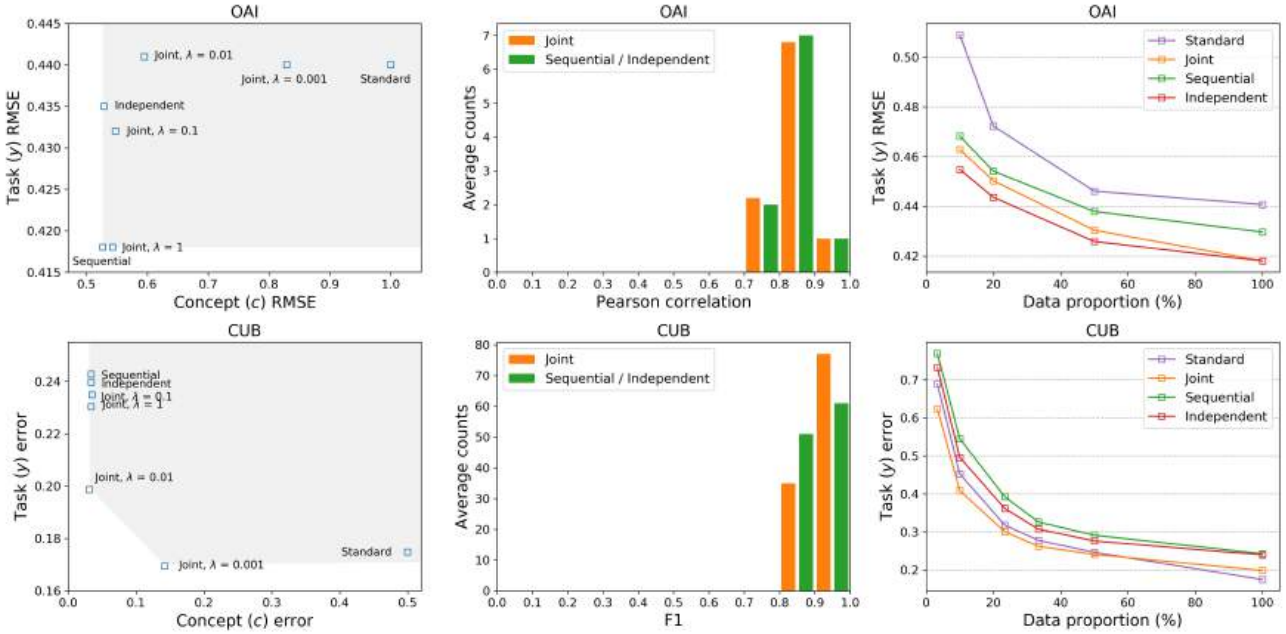


Figure 2. **Left**: The shaded regions show the optimal frontier between task vs. concept error. We find little trade-off; models can do well on both task and concept prediction. For standard models, we plot the concept error of the mean predictor (OAI) or random predictor (CUB). **Mid**: Histograms of how accurate individual concepts are, averaged over multiple random seeds. In our tasks, each individual concept can be accurately predicted by bottleneck models. **Right**: Data efficiency curves. Especially on OAI, bottleneck models can achieve the same task accuracy as standard models with many fewer training points.

in Section 1, low concept error suggests that the model's concepts are aligned with the true concepts, which in turn suggests that we might intervene effectively on them; we will explore this in Section 6.

Overall, we do not observe a tradeoff between high task accuracy and high concept accuracy: pulling the bottleneck layer towards the concepts c does not substantially affect the model's ability to predict y in our tasks, even when the bottleneck is trained jointly. We illustrate this in Figure 2-Left, which plots the task vs. concept errors of each model.

Additional baselines. We ran two further baselines to determine if the bottleneck architecture impacted model performance. First, standard models in the literature do not use concept bottlenecks, so we trained a variant of the standard model without the bottleneck layer (directly using a ResNet-18 or Inception-v3 model to predict $x \to y$); this performed similarly to the standard bottleneck model ("Standard, no bottleneck" in Table 1). Second, we tested a typical multi-task setup using an auxiliary loss to encourage the activations of the last layer to be predictive of the concepts c, hyperparameter searching across different weightings of this auxiliary loss. These models also performed comparably ("Multitask" in Table 1), but since they do not support concept interventions, we focus on comparing standard vs. concept bottleneck models in the rest of the paper.

Data efficiency. Another way to benchmark different models is by measuring data efficiency, i.e., how many training points they need for a desired level of accuracy. To study this, we subsampled the training and validation data and retrained each model (details in Appendix B.4). Concept bottleneck models are particularly effective on OAI: the sequential bottleneck model with $\approx 25\%$ of the full dataset performs similarly to the standard model. On CUB, the joint bottleneck and standard models are more accurate throughout, with the joint model slightly more accurate in lower data regimes (Figure 2-Right).

A drawback of concept bottleneck models is that they require annotated concepts at training time. However, if the set of concepts are good enough, then fewer training examples might be required to achieve a desired accuracy level (as in OAI). This allows model developers to trade off the cost of acquiring more detailed annotations against the cost of acquiring new training examples, which can be helpful when new training examples are expensive to acquire, e.g., in medical settings where adding training points might entail invasive/expensive procedures on patients, but the incremental cost in asking a doctor to add annotations to data points that they already need to look at might be low.

5. Benchmarking post-hoc concept analysis

Concept bottleneck models are trained to have a bottleneck layer that aligns component-wise with the human-specified concepts c. For any test input x, we can read out predicted concepts directly from the bottleneck layer, as well as intervene on concepts by manipulating the predicted concepts $\hat{c}$ and inspecting how the final prediction $\hat{y}$ changes. This enables explanations like "if the model did not think the joint space was too narrow for this patient, then it would not have predicted severe arthritis". An alternative approach to interpreting models in terms of concepts is post-hoc analysis: take an existing model trained to directly predict $x \to y$ without any concepts, and use a probe to recover the known concepts from the model's activations. For example, Bau et al. (2017) measure the correlation of individual neurons with concepts, while Kim et al. (2018) use a linear probe to predict concepts with linear combinations of neurons.

Post-hoc analysis does not enable interventions on concepts: even if we find a linear combination of neurons that predicts a concept well, it is unclear how to modify the model's activations to change what it thinks of that concept alone. Without this ability to intervene, interpretations in terms of concepts is suggestive but fraught: even if we can say that "the model thinks the joint space is narrow", it is hard to test if that actually affects its final prediction. This is an important limitation of post-hoc interpretation. Nonetheless, setting this point aside for a moment, post-hoc interpretations require high concept accuracy. We therefore evaluate how accurately probes can predict concepts post-hoc.

Following Kim et al. (2018), we trained a linear probe to predict each concept from the layers of the standard model (see Appendix B). We found that these linear probes have lower concept accuracy compared to simply reading concepts out from a bottleneck model (Table 2). On OAI, the best-performing linear probe achieved an average concept RMSE of 0.68, vs. 0.53 in the bottleneck models; average Pearson correlation dropped to 0.72 from 0.84. On CUB, the linear probe achieved an average concept error of 0.09 instead of 0.03; average F1 score dropped to 0.77 from 0.92.

We also tested if we could predict concepts post-hoc from models designed to learn an interpretable mapping from $x \to y$. Specifically, we evaluated self-explaining neural networks (SENN) (Melis & Jaakkola, 2018). As with standard models, SENN does not use any pre-specified concepts; it learns an input representation encouraged to be interpretable through diversity and smoothness constraints. However, linear probes on SENN also had lower concept accuracy on OAI (0.68 concept RMSE; see Appendix B).[4]

[4]We were unable to run SENN on CUB because the default implementation was too memory-intensive; CUB has many more classes/concepts than the tasks SENN was originally used for.

The comparative difficulty in predicting concepts post-hoc suggests that if we have prior knowledge of what concepts practitioners would use, then it helps to directly train models with these concepts instead of hoping to recover them from a model trained without knowledge of these concepts. See Chen et al. (2020) for a related discussion.

6. Test-time intervention

The ability to intervene on concept bottleneck models enables human users to have richer interactions with them. For example, if a radiologist disagrees with a model's prediction, she would not only be able to inspect the predicted concepts, but also simulate how the model would respond to changes in those predicted concepts. This kind of *test-time intervention* can be particularly useful in high-stakes settings like medicine, or in other settings where it is easier for users to identify the concepts c (e.g., wing color) than the target y (exact species of bird).

We envision that in practice, domain experts interacting with the model could intervene to "fix" potentially incorrect concepts. To study this setting, we use an oracle that can query the true value of any concept for a test input. Figure 3 shows several examples of interventions that lead to the model making a correct prediction.

6.1. Intervening on OAI

On OAI, we intervene on a concept by simply replacing the model's corresponding predicted concept $\hat{c}_j$ with its true value c_j (Figure 3-Left). To simplify testing multiple interventions, we use an input-independent ordering over concepts computed from the held-out validation set (i.e., we always intervene on some concept c_{i_1} first, followed by c_{i_2}, etc.; see Appendix B).

Test-time intervention on OAI significantly improved task accuracy: e.g., querying for just 2 concepts reduces task RMSE from >0.4 to ≈ 0.3 (Figure 4-Left). Neural networks similar to ours have been previously noted to be comparable with individual radiologist performance on grading KLG (compared to the consensus grade, which we use as ground truth; see Tiulpin et al. (2018); Pierson et al. (2019)). As the concept values used for intervention mostly come from a single radiologist instead of a consensus reading (see Appendix A), these results hint that a single radiologist collaborating with bottleneck models might be able to outperform either the radiologist or model alone, though more careful human studies would be needed to evaluate that.

The independent bottleneck achieved better test error when all $k = 10$ concepts are replaced than the sequential or joint bottlenecks (Figure 4-Left). This is expected; when all concepts are replaced, the $x \to c$ part of the model is irrelevant, and all that matters is the $c \to y$ part. Recall that in the

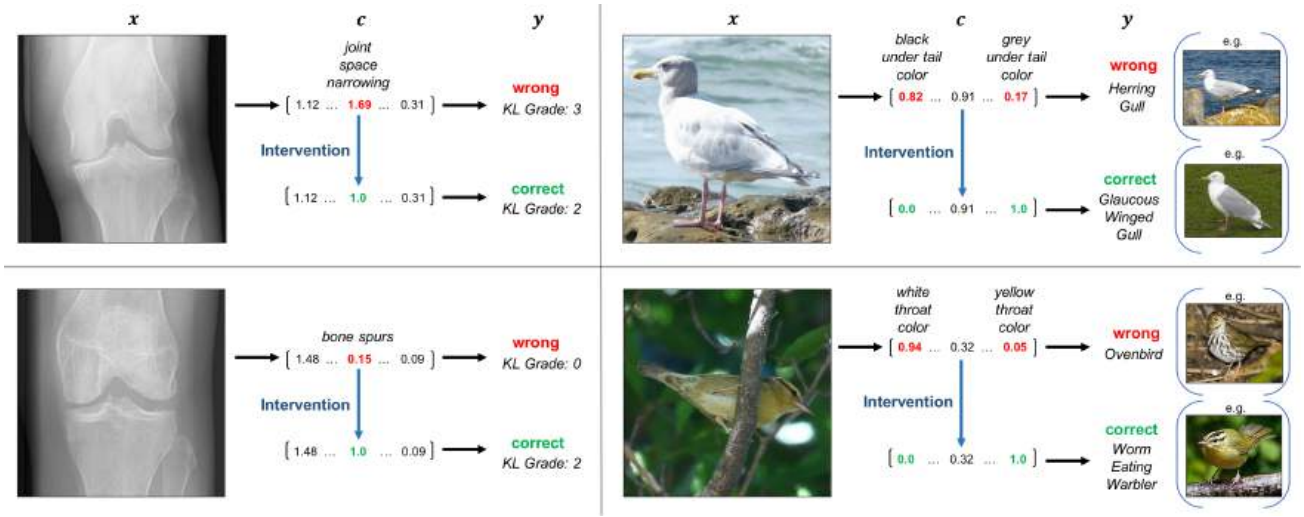


Figure 3. Successful examples of test-time intervention, where intervening on a single concept corrects the model prediction. Here, we show examples from independent bottleneck models. **Right**: For CUB, we intervene on concept groups instead of individual binary concepts. The sample birds on the right illustrate how the intervened concept distinguishes between the original and new predictions.

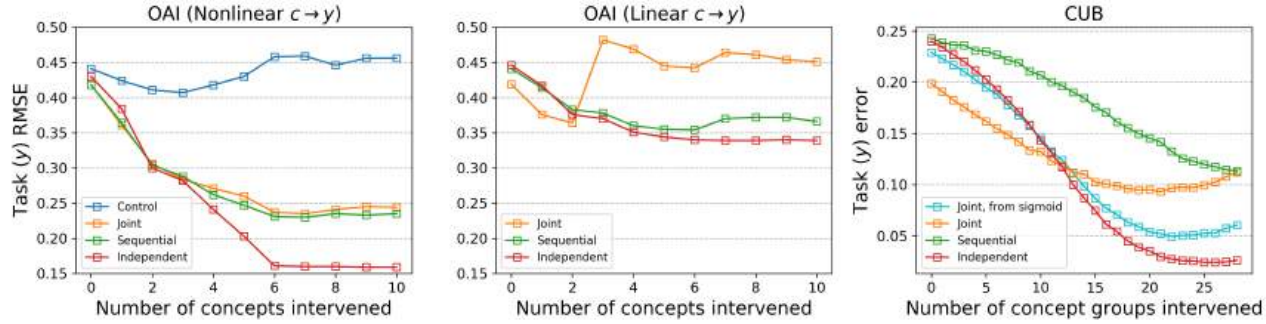


Figure 4. Test-time intervention results. **Left**: Intervention substantially improves task accuracy, except for the control model, which is a joint model that heavily prioritizes label accuracy over concept accuracy. **Mid**: Replacing $c \to y$ with a linear model degrades effectiveness. **Right**: Intervention improves task accuracy except for the joint model. Connecting $c \to y$ to probabilities rescues intervention but degrades normal accuracy.

independent bottleneck, $c \to y$ is trained using the true c, which is what we replace the predicted concepts $\hat{c}$ with. In contrast, in the sequential and joint models, $c \to y$ is trained using the predicted $\hat{c}$, which in general will have a different distribution from the true c. This example illustrates a trade-off between intervenability and task accuracy: the independent bottleneck performs worse without interventions (Table 1), but better with interventions.

To better understand what influences intervention effectiveness, we ran two ablations. First, we found that intervention can fail in joint models when λ is too small (recall that the smaller λ is, the more we prioritize fitting y over c in training). Specifically, the joint model with $\lambda = 0.01$ learned a concept representation that was not as well-aligned with the true concepts, and replacing $\hat{c}$ with the true c at test time slightly *increased* test error ("control" model in Figure 4-Left). Second, we changed the $c \to y$ model from the 3-layer MLP used throughout the paper to a single linear layer. Test-time intervention was less effective here compared to the non-linear counterparts (Figure 4-Mid), even though task and concept accuracies were similar before intervention (concept RMSEs of the sequential and independent models are not even affected by the change in $c \to y$).

Altogether, these results suggest that task and concept accuracies alone are insufficient for determining how effective test-time intervention will be on a model. Different inductive biases in different models control how effectively they can handle distribution shifts from $\hat{c} \to y$ (pre-intervention) to $c \to y$ (post-intervention). Even without this distribution shift, as in the case of the linear vs. non-linear independent bottlenecks, the expressivity of $c \to y$ has a large effect on intervention effectiveness. Moreover, it is possible that the average concept accuracy masks differences in individual concept accuracies that influence these results.

6.2. Intervening on CUB

Intervention on CUB is complicated by the fact that it is classification instead of regression. Recall from Section 3 that for sequential and joint bottleneck classifiers, we connect $c \to y$ to the logits $\hat{\ell} = \hat{g}(x)$. To intervene on a concept $\hat{c}_j$, we therefore cannot directly copy over the true c_j. Instead, we need to alter the logits $\hat{\ell}_j$ such that $P(\hat{c}_j = 1) = \sigma(\hat{\ell}_j)$ is close to the true c_j. Concretely, we intervene on $\hat{c}_j$ by setting $\hat{\ell}_j$ to the 5th (if $c_j = 0$) or 95th (if $c_j = 1$) percentile of $\hat{\ell}_j$ over the training distribution.

Another difference is that for CUB, we group related concepts and intervene on them together. This is because many of the concepts encode the same underlying property, e.g., $c_1 = 1$ if the wing is red, $c_2 = 1$ if the wing is black, etc. We assume that the human (oracle) returns the true wing color in a single query, instead of only answering yes/no questions about the wing color; see Figure 4-Right.

An important caveat is that we use denoised class-level concepts in the CUB dataset (Section 4.1). To avoid unrealistic scenarios where a bird part is not visible in the image but we still 'intervene' on it, we only replace a concept value with the true concept value if that concept is actually visible in the image (visibility information is included in the dataset). The results here are nonetheless still optimistic, because they assume that human experts do not make mistakes in identifying concepts and that birds of the same species always share the same concept values.

Test-time intervention substantially improved accuracy on CUB bottleneck models (Figure 4-Right), though it took intervention on several concept groups to see a large gain. For simplicity, we queried concept groups in random order, which means that many queries were probably irrelevant for any given test example.

Test-time intervention was more effective on independent bottleneck models than on the sequential and joint models (Figure 4-Right). We hypothesize that this is partially due to the ad hoc fashion in which we set logits to the 5th or 95th percentiles for the latter models. To study this, we trained a joint bottleneck with the same task-concept tradeoff λ but with $c \to y$ connected to the probabilities $P(\hat{c}_j = 1) = \sigma(\hat{\ell}_j)$ instead of the logits $\hat{\ell}_j$. This model had a higher task error of 0.224 vs. 0.199 with the normal joint model; we suspect that the squashing from the sigmoid makes optimization harder. However, test-time intervention worked better ("Joint, from sigmoid" vs. "Joint" in Figure 4-Right), and it is more straightforward as we can directly edit $\hat{c}$. This poses the question of how to effectively intervene in the classification setting while maintaining the computational advantages of avoiding the sigmoid in the $c \to y$ connection.

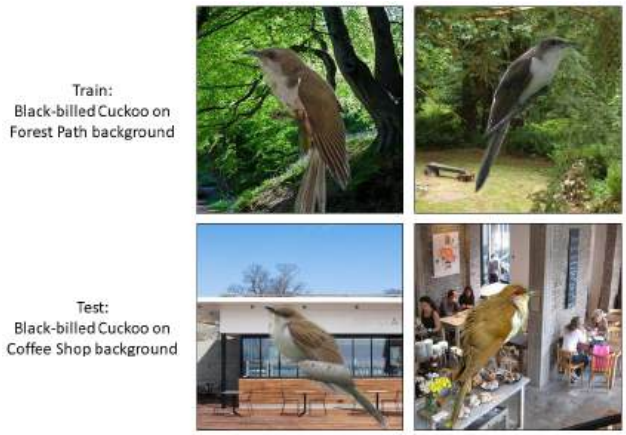


Figure 5. We change the image backgrounds associated with each class from train to test time (illustrated above for a single class).

Table 3. Task and concept error with background shifts. Bottleneck models have substantially lower task error than the standard model.

MODEL	y ERROR	c ERROR
STANDARD	0.627±0.013	-
JOINT	0.482±0.018	0.069±0.002
SEQUENTIAL	0.496±0.009	0.072±0.002
INDEPENDENT	0.482±0.008	0.072±0.002

7. Robustness to background shifts

Finally, we investigate if concept bottleneck models can be more robust than standard models to spurious correlations (e.g., the background) that hold in the training distribution but not the test distribution. Whether bottleneck models are more robust depends on the choice of the set of concepts c and the shifts considered; a priori, we do not expect that an arbitrary set of concepts c will lead to a more robust model.

We constructed a variant of the CUB dataset where the target y is spuriously correlated with image background in the training set. Specifically, we cropped each bird out of its original background (using segmentation masks from the original dataset) and onto a new background from the Places dataset (Zhou et al., 2017), with each bird class (species) assigned to a unique and randomly-selected category of places. At test time, we shuffle this mapping, so each class is associated with a different category of places. For example, at training time, all robins might be pictured against the sky, but at test time they might all be on grassy plains (Figure 5).

As images from each class now share common background features, standard models leverage this spurious correlation and consequently fail on the shifted test set (Table 3). Concept models do better as they rely less on background features, since each concept is shared among multiple bird classes and thus appears in training data points that span multiple background types, reducing the correlation between

the concept and the background. This toy experiment shows that concept bottleneck models can be more robust to spurious correlations when the target y is more correlated with training data artifacts compared to the concepts c.

8. Discussion

Concept bottleneck models can compete on task accuracy while supporting intervention and interpretation, allowing practitioners to reason about these models in terms of high-level concepts they are familiar with, and enabling more effective human-model collaboration through test-time intervention. We believe that these models can be promising in settings like medicine, where the high stakes incentivize human experts to collaborate with models at test time, and where the tasks are often normatively defined with respect to a set of standard concepts (e.g., "osteoarthritis is marked by the presence of bone spurs"). A flurry of recent papers have used similar human concepts for post-hoc interpretation of medical and other scientific ML models, e.g., Graziani et al. (2018) for breast cancer histopathology; Clough et al. (2019) for cardiac MRIs; and Sprague et al. (2019) for meteorology (storm prediction). We expect that concept bottleneck models can be applied directly to similar settings. Below, we discuss several directions for future work.

Learning concepts. In tasks that are not normatively defined, we can learn the right concepts by interactively querying humans. For example, Cheng & Bernstein (2015) asked crowdworkers to generate concepts to differentiate between adaptively-chosen pairs of examples, and used those concepts to train models to recognize the artist of a painting, tell honest from deceptive reviews, and identify popular jokes. Similar methods can also be used to refine existing concepts and make them more discriminative (Duan et al., 2012).

Side channel from $x \to y$. We can also account for having an incomplete set of concepts by adding a direct side channel from $x \to y$ to a bottleneck model. This is equivalent to using the concepts as auxiliary features, and as discussed in Section 2, has the drawback that we cannot cleanly intervene on a single concept, since the $x \to y$ connection might also be implicitly reasoning about that concept. Devising approaches to mitigate this issue would allow concept models to have high task accuracy even with an incomplete set of concepts; for example, one might consider carefully regularizing the $x \to y$ connection or using some sort of adversarial loss to prevent it from using existing concepts.

Theoretically analyzing concept bottlenecks. A better understanding of when and why concept bottlenecks improve task accuracy can inform how we collect concepts or design the architecture of bottleneck models. As an example of what this could entail, we sketch an analysis of a simple well-specified linear regression setting, where we assume that the input $x \in \mathbb{R}^d$ is normally distributed, and that the concepts $c \in \mathbb{R}^k$ and the target $y \in \mathbb{R}$ are noisy linear transformations of x and c respectively. We compared an independent bottleneck model (two linear regression problems for $x \to c$ and $c \to y$) to a standard model (a single linear regression problem) by deriving the ratio of their excess mean-squared-errors as the number of training points n goes to infinity:

$$\frac{\text{Excess error for indp bottleneck model}}{\text{Excess error for standard model}} \leq \frac{\frac{k}{d}\sigma_Y^2 + \sigma_C^2}{\sigma_Y^2 + \sigma_C^2},$$

where σ_C^2 and σ_Y^2 are the variances of the noise in the concepts c and target y, respectively. See Appendix C for a formal statement and proof. Note that the asymptotic relative excess error is small when $\frac{k}{d}$ is small and $\sigma_Y^2 \gg \sigma_C^2$, suggesting that concept bottleneck models can be particularly effective when the number of concepts is much smaller than the input dimension and when the concepts have relatively low noise compared to the target.

Intervention effectiveness. Our exploration of the design space of concept bottleneck models showed that the training method (independent, sequential, joint) and choice of architecture have a large influence not just on task and concept accuracies, but also on how effective interventions are. This poses several open questions, for example: What factors drive the effectiveness of test-time interventions? Does concept accuracy suffice for comparing the interpretability of concept bottleneck models, or is intervention effectiveness more important? Could adaptive strategies that query for the concepts that maximize expected information gain on a particular test example make interventions more effective? Finally, how might we have models learn from interventions to avoid making similar mistakes in the future?

Reproducibility

The code for replicating our experiments is available on GitHub at `https://github.com/yewsiang/ConceptBottleneck`. An executable version of the CUB experiments in this paper is on CodaLab at `https://worksheets.codalab.org/worksheets/0x362911581fcd4e048ddfd84f47203fd2`. The post-processed CUB+Places dataset can also be downloaded at that link. While we are unable to release the OAI dataset publicly, an application to access the data can be made at `https://nda.nih.gov/oai/`.

Acknowledgements

We are grateful to Jesse Mu, Justin Cheng, Kensen Shi, Michael Bernstein, Rui Shu, Sendhil Mullainathan, Shyamal Buch, Yair Carmon, Ziad Obermeyer, and our anonymous reviewers for helpful advice. PWK was supported by a Facebook PhD Fellowship. YST was supposed by an IMDA Singapore Digital Scholarship. SM was supported by an NSF Graduate Fellowship. EP was supported by a Hertz Fellowship. Other funding came from the PECASE Award. Toyota Research Institute ("TRI") provided funds to assist the authors with their research but this article solely reflects the opinions and conclusions of its authors and not TRI or any other Toyota entity.

References

Bau, D., Zhou, B., Khosla, A., Oliva, A., and Torralba, A. Network dissection: Quantifying interpretability of deep visual representations. In *Computer Vision and Pattern Recognition (CVPR)*, pp. 6541–6549, 2017.

Bengio, Y., Courville, A., and Vincent, P. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35 (8):1798–1828, 2013.

Bucher, M., Herbin, S., and Jurie, F. Semantic bottleneck for computer vision tasks. In *Asian Conference on Computer Vision*, pp. 695–712, 2018.

Bunescu, R. C. and Mooney, R. J. A shortest path dependency kernel for relation extraction. In *Empirical Methods in Natural Language Processing (EMNLP)*, pp. 724–731, 2005.

Chen, X., Duan, Y., Houthooft, R., Schulman, J., Sutskever, I., and Abbeel, P. InfoGAN: Interpretable representation learning by information maximizing generative adversarial nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

Chen, Z., Bei, Y., and Rudin, C. Concept whitening for interpretable image recognition. *arXiv preprint arXiv:2002.01650*, 2020.

Cheng, J. and Bernstein, M. S. Flock: Hybrid Crowd-Machine learning classifiers. In *Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work & Social Computing*, pp. 600–611, 2015.

Clough, J. R., Oksuz, I., Puyol-Antón, E., Ruijsink, B., King, A. P., and Schnabel, J. A. Global and local interpretability for cardiac MRI classification. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pp. 656–664, 2019.

Cui, Y., Song, Y., Sun, C., Howard, A., and Belongie, S. Large scale fine-grained categorization and domain-specific transfer learning. In *Computer Vision and Pattern Recognition (CVPR)*, pp. 4109–4118, 2018.

Duan, K., Parikh, D., Crandall, D., and Grauman, K. Discovering localized attributes for fine-grained recognition. In *Computer Vision and Pattern Recognition (CVPR)*, pp. 3474–3481, 2012.

Fauw, J. D., Ledsam, J. R., Romera-Paredes, B., Nikolov, S., Tomasev, N., Blackwell, S., Askham, H., Glorot, X., O'Donoghue, B., Visentin, D., et al. Clinically applicable deep learning for diagnosis and referral in retinal disease. *Nature Medicine*, 24(9):1342–1350, 2018.

Fong, R. C. and Vedaldi, A. Interpretable explanations of black boxes by meaningful perturbation. In *International Conference on Computer Vision (ICCV)*, pp. 3429–3437, 2017.

Ghorbani, A., Wexler, J., Zou, J. Y., and Kim, B. Towards automatic concept-based explanations. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 9277–9286, 2019.

Goyal, Y., Shalit, U., and Kim, B. Explaining classifiers with causal concept effect (CaCE). *arXiv preprint arXiv:1907.07165*, 2019.

Graziani, M., Andrearczyk, V., and Müller, H. Regression concept vectors for bidirectional explanations in histopathology. In *Understanding and Interpreting Machine Learning in Medical Image Computing Applications*, pp. 124–132, 2018.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Computer Vision and Pattern Recognition (CVPR)*, 2016.

Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S., and Lerchner, A. beta-vae: Learning basic visual concepts with a constrained variational framework. In *International Conference on Learning Representations (ICLR)*, 2017.

Huang, S., Xu, Z., Tao, D., and Zhang, Y. Part-stacked CNN for fine-grained visual categorization. In *Computer Vision and Pattern Recognition (CVPR)*, pp. 1173–1182, 2016.

Kellgren, J. and Lawrence, J. Radiological assessment of osteo-arthrosis. *Annals of the Rheumatic Diseases*, 16(4), 1957.

Kim, B., Wattenberg, M., Gilmer, J., Cai, C., Wexler, J., Viegas, F., et al. Interpretability beyond feature attribution: Quantitative testing with concept activation vectors

(tcav). In *International Conference on Machine Learning (ICML)*, pp. 2668–2677, 2018.

Kumar, N., Berg, A. C., Belhumeur, P. N., and Nayar, S. K. Attribute and simile classifiers for face verification. In *International Conference on Computer Vision (ICCV)*, pp. 365–372, 2009.

Lampert, C. H., Nickisch, H., and Harmeling, S. Learning to detect unseen object classes by between-class attribute transfer. In *Computer Vision and Pattern Recognition (CVPR)*, pp. 951–958, 2009.

Losch, M., Fritz, M., and Schiele, B. Interpretability beyond classification output: Semantic bottleneck networks. *arXiv preprint arXiv:1907.10882*, 2019.

Melis, D. A. and Jaakkola, T. Towards robust interpretability with self-explaining neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 7775–7784, 2018.

Murty, S., Koh, P. W., and Liang, P. ExpBERT: Representation engineering with natural language explanations. In *Association for Computational Linguistics (ACL)*, 2020.

Nevitt, M., Felson, D. T., and Lester, G. The Osteoarthritis Initiative. *Cohort study protocol*, 2006.

O'Shaughnessy, M., Canal, G., Connor, M., Davenport, M., and Rozell, C. Generative causal explanations of black-box classifiers. *arXiv preprint arXiv:2006.13913*, 2020.

Pearl, J. *Causality: Models, Reasoning and Inference*, volume 29. Springer, 2000.

Pierson, E., Cutler, D., Leskovec, J., Mullainathan, S., and Obermeyer, Z. Using machine learning to understand racial and socioeconomic differences in knee pain. *NBER Machine Learning and Healthcare Conference*, 2019.

Sprague, C., Wendoloski, E. B., and Guch, I. Interpretable AI for deep learning-based meteorological applications. In *99th American Meteorological Society Annual Meeting*, 2019.

Sutton, C. and McCallum, A. Joint parsing and semantic role labeling. In *Computational Natural Language Learning (CoNLL)*, 2005.

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., and Wojna, Z. Rethinking the Inception architecture for computer vision. In *Computer Vision and Pattern Recognition (CVPR)*, pp. 2818–2826, 2016.

Tiulpin, A., Thevenot, J., Rahtu, E., Lehenkari, P., and Saarakkala, S. Automatic knee osteoarthritis diagnosis from plain radiographs: A deep learning-based approach. *Scientific Reports*, 8(1):1–10, 2018.

Wah, C., Branson, S., Welinder, P., Perona, P., and Belongie, S. The Caltech-UCSD Birds-200-2011 dataset. Technical report, California Institute of Technology, 2011.

Yi, K., Wu, J., Gan, C., Torralba, A., Kohli, P., and Tenenbaum, J. Neural-symbolic vqa: Disentangling reasoning from vision and language understanding. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 1031–1042, 2018.

Zelenko, D., Aone, C., and Richardella, A. Kernel methods for relation extraction. *Journal of Machine Learning Research*, 3(0):1083–1106, 2003.

Zhou, B., Lapedriza, A., Khosla, A., Oliva, A., and Torralba, A. Places: A 10 million image database for scene recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(6):1452–1464, 2017.

Zhou, B., Sun, Y., Bau, D., and Torralba, A. Interpretable basis decomposition for visual explanation. In *European Conference on Computer Vision (ECCV)*, pp. 119–134, 2018.